

Notes: <https://projects.100xdevs.com/tracks/Auth/auth-1>

Github:

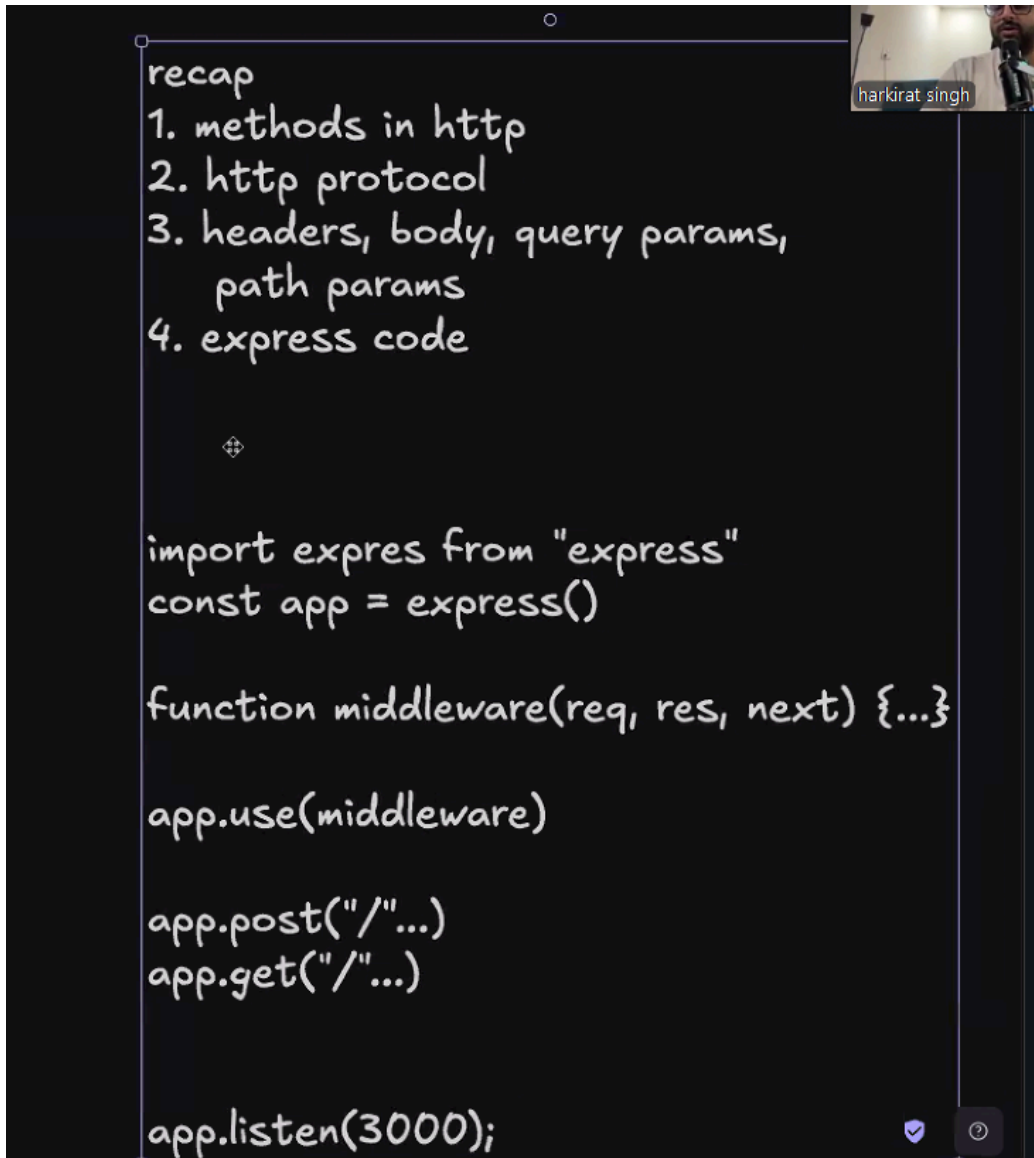
14th-march-notes-app-unauth :

<https://github.com/100xdevs-bootcamp-1/14th-march-notes-app-unauth>

14th-march-notes-app-auth:

<https://github.com/100xdevs-bootcamp-1/14th-march-notes-app-auth>

1. Last Class Recap



```
recap
1. methods in http
2. http protocol
3. headers, body, query params,
   path params
4. express code

import expres from "express"
const app = express()

function middleware(req, res, next) {...}

app.use(middleware)

app.post("/")...
app.get("/")...

app.listen(3000);
```

2. Today we will read Authentication with JWT localstorage

The diagram shows a hierarchy where 'cors' points to 'authentication', which then branches into 'jwt + localstorage' and 'cookies'.

```
graph TD
  cors --> authentication
  authentication --> jwt["jwt + localstorage"]
  authentication --> cookies
```

recap

1. methods in http
2. http protocol
3. headers, body, query params, path params
4. express code

```
import express from "express"
const app = express()

function middleware(req, res, next) {...}

app.use(middleware)

app.post("/")
app.get("/")

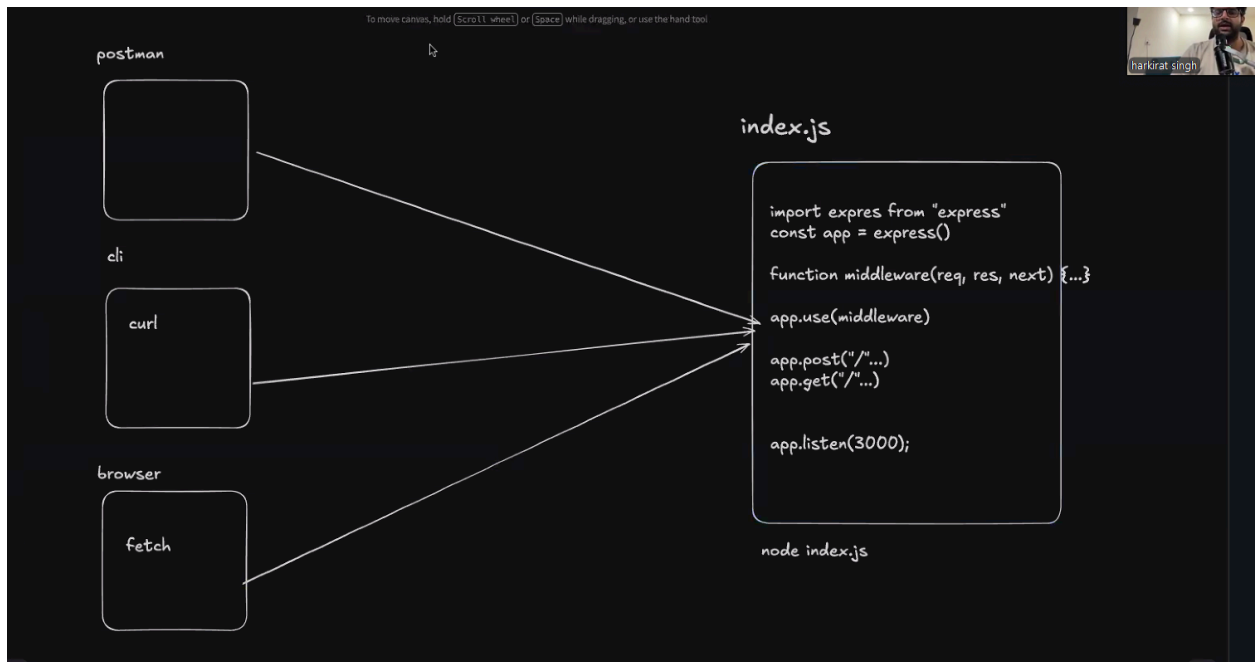
app.listen(3000);
```

3. Today Content

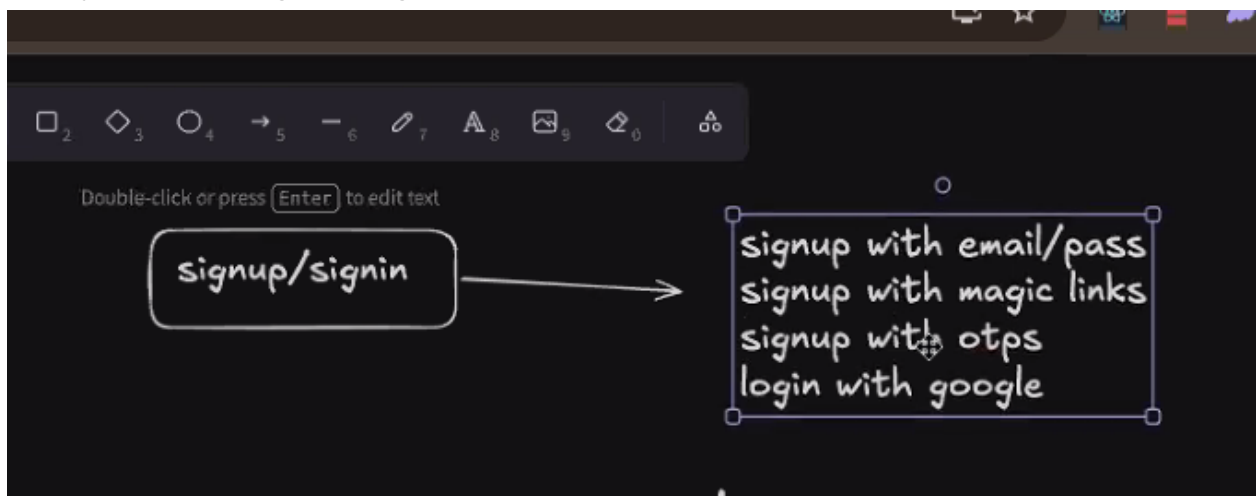
The screenshot shows the 'Authentication (1 / 8)' track on the DailyCode website. A table of contents is visible, with the second item, 'Authentication using jwt + localstorage', circled in blue.

Jump To	Prev	Next	Switch to New
1 - What is authentication?			
2 - Authentication using jwt + localstorage			
3 - Authentication using cookies (Part 1)			
4 - Authentication with cookies (Part 2)			
5 - Properties of cookies			
6 - Example in express (Backend)			
7 - Frontend in React			
8 - Frontend from express			

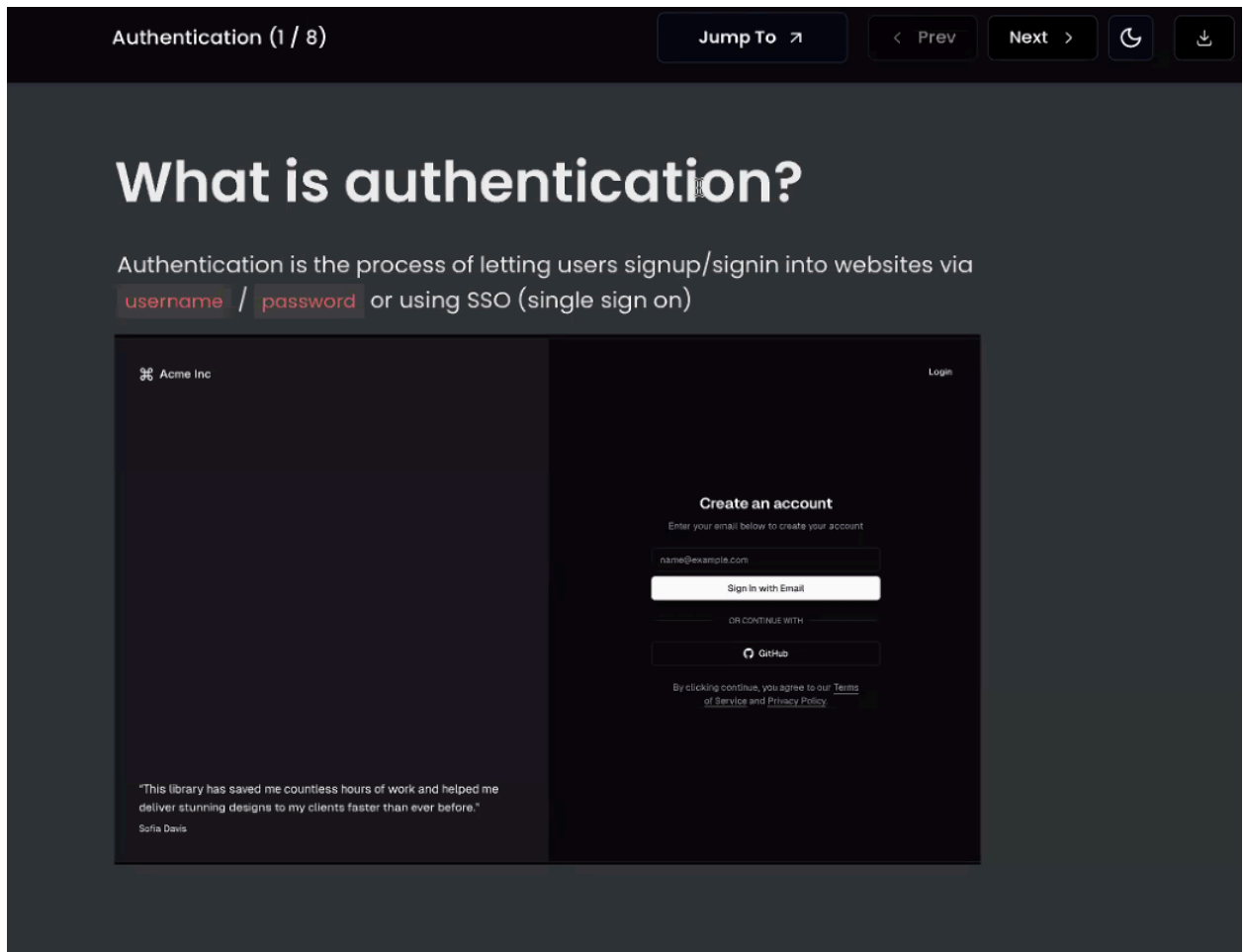
4. How frontend and backend works



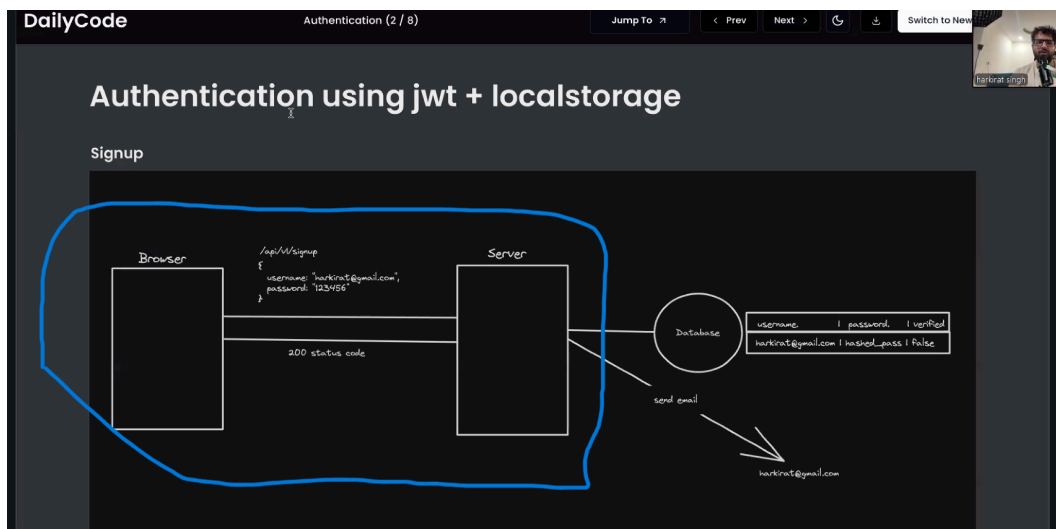
5. Every website sign-in/sign-up look like below



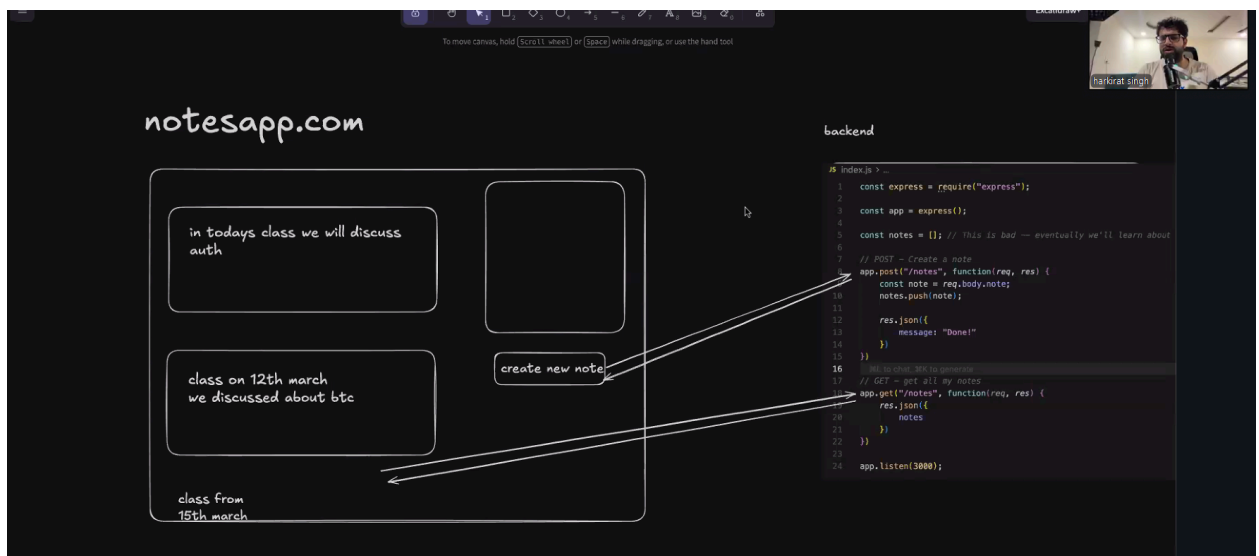
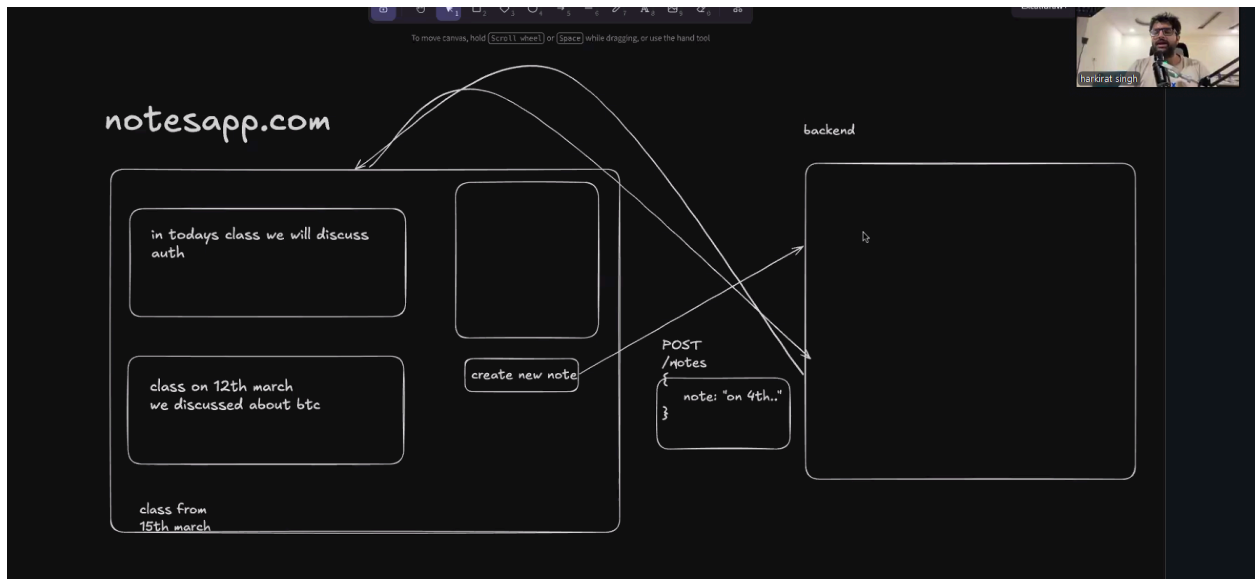
6. What is authentication



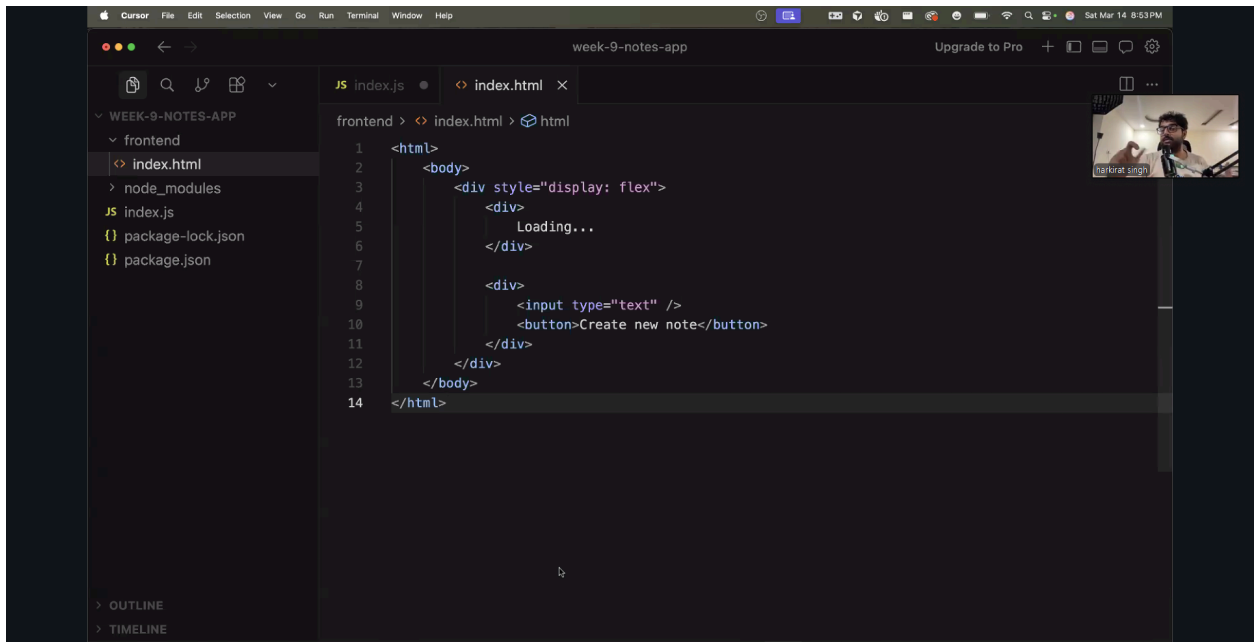
7. Authentication with jwt localStorage(Today we will make in-memory database)



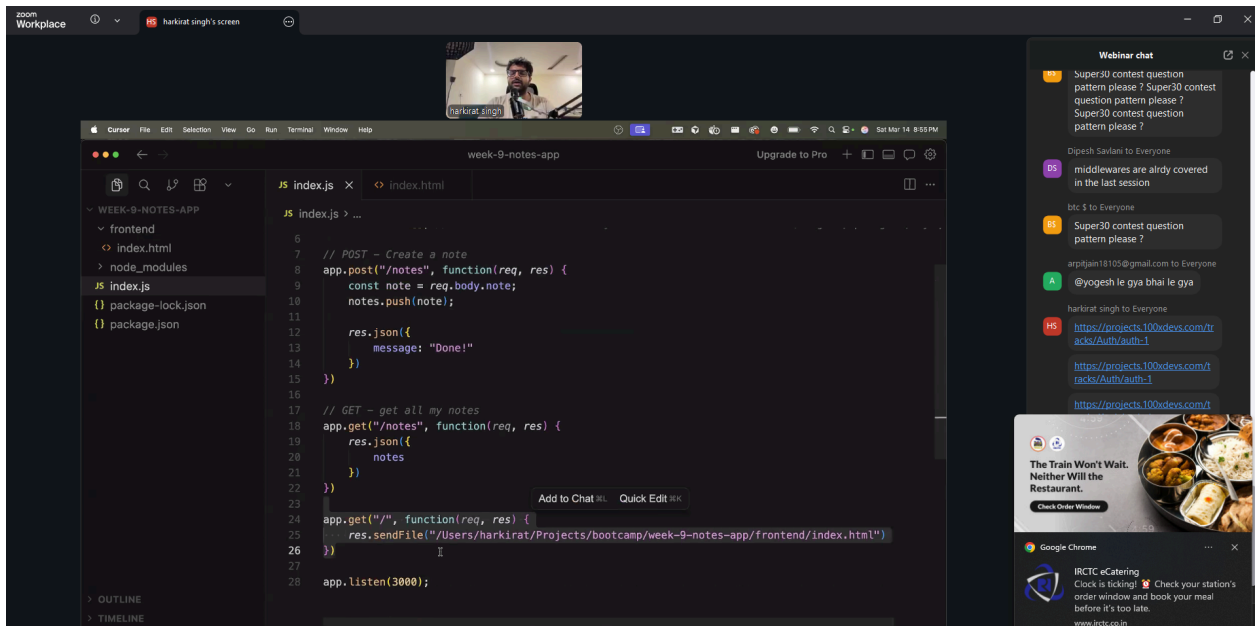
8. Lets first make high level frontend backend architecture



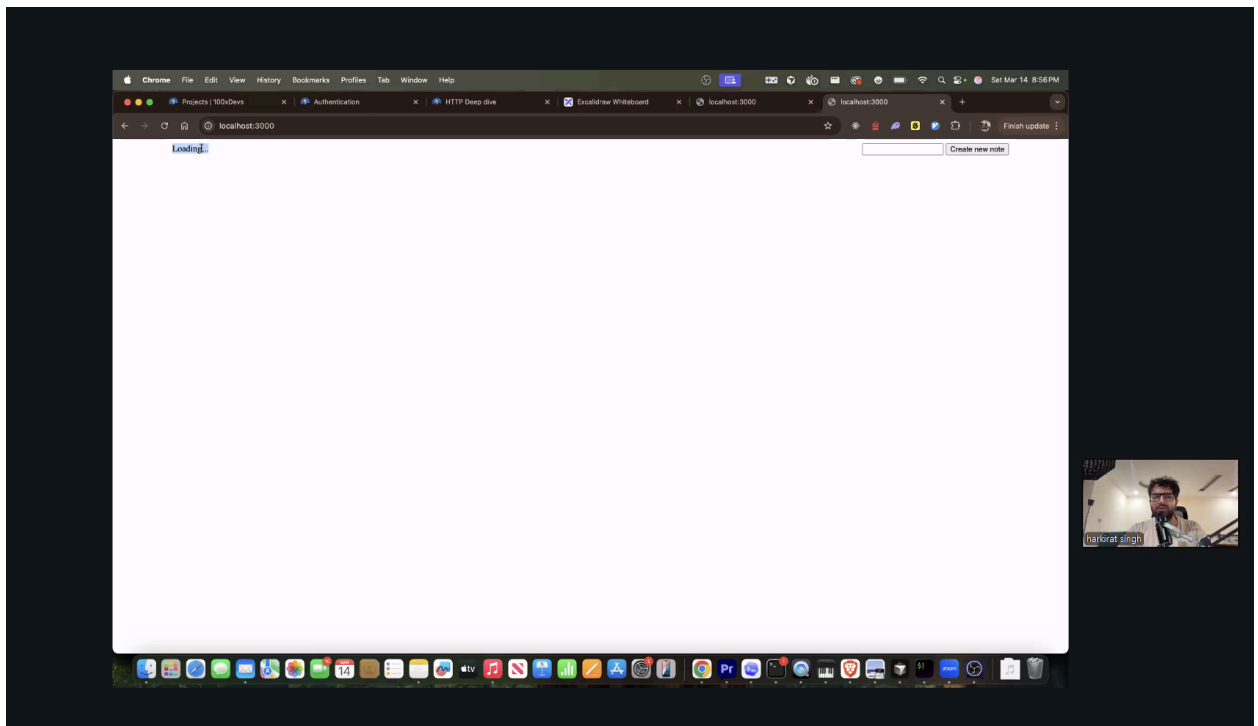
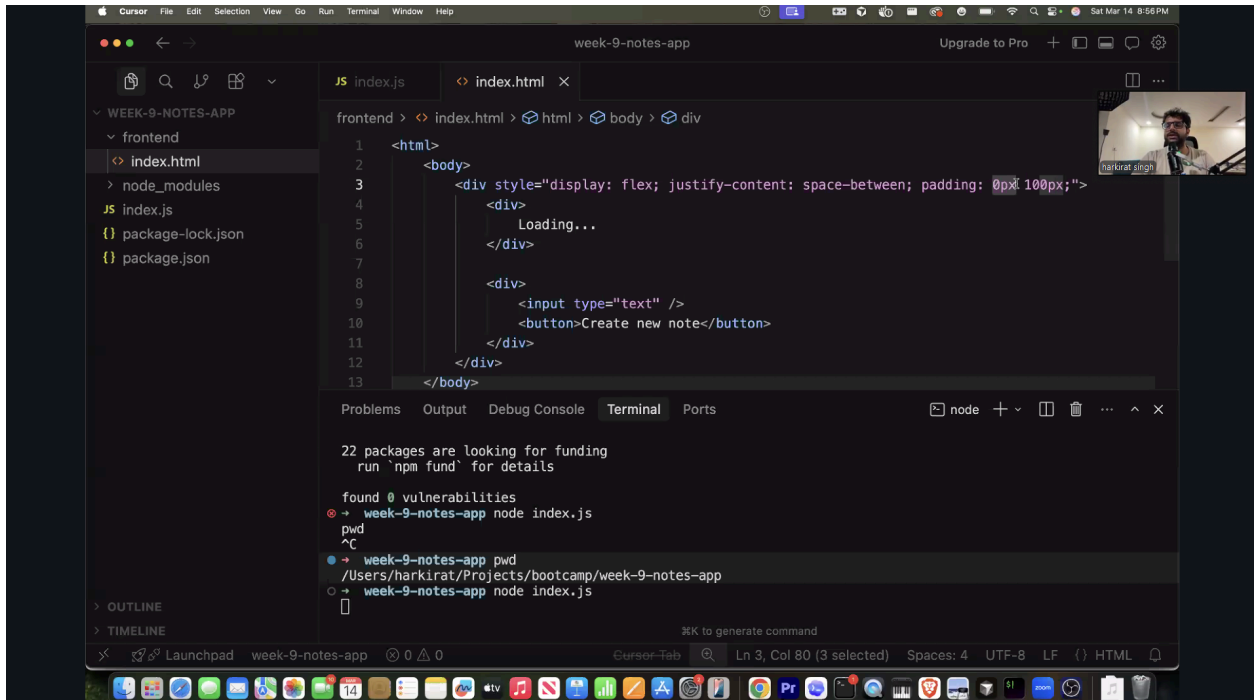
Step1: Initially when I will click domain then it render a page -> get request (we show Loader until get api fetch data)



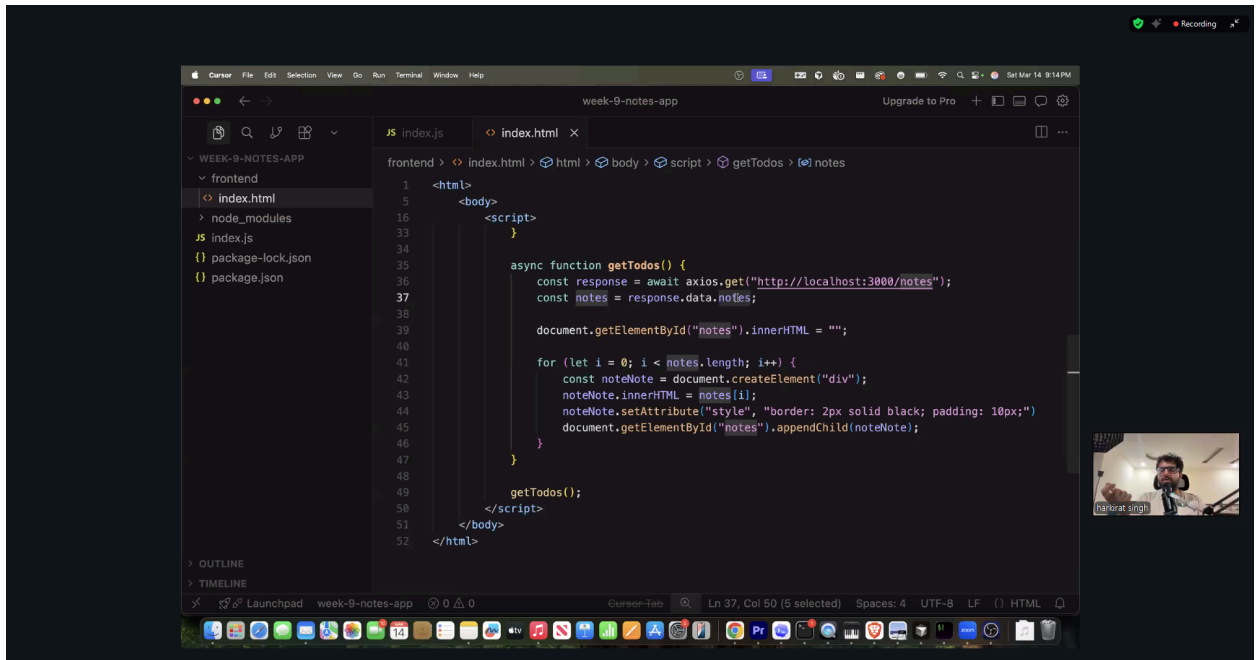
```
1 <html>
2   <body>
3     <div style="display: flex">
4       <div>
5         Loading...
6       </div>
7
8       <div>
9         <input type="text" />
10        <button>Create new note</button>
11      </div>
12    </div>
13  </body>
14 </html>
```



```
6 // ...
7
8 // POST - Create a note
9 app.post("/notes", function(req, res) {
10   const note = req.body.note;
11   notes.push(note);
12
13   res.json({
14     message: "Done!"
15   })
16 })
17
18 // GET - get all my notes
19 app.get("/notes", function(req, res) {
20   res.json({
21     notes
22   })
23 })
24
25 app.get("/", function(req, res) {
26   res.sendFile(__dirname + "/Users/harkirat/Projects/bootcamp/week-9-notes-app/frontend/index.html")
27 })
28
29 app.listen(3000);
```



- Get todos frontend code(GET request)
fetch() => browser specific provider
axios() => library that build on top of fetch function and it is cleaner

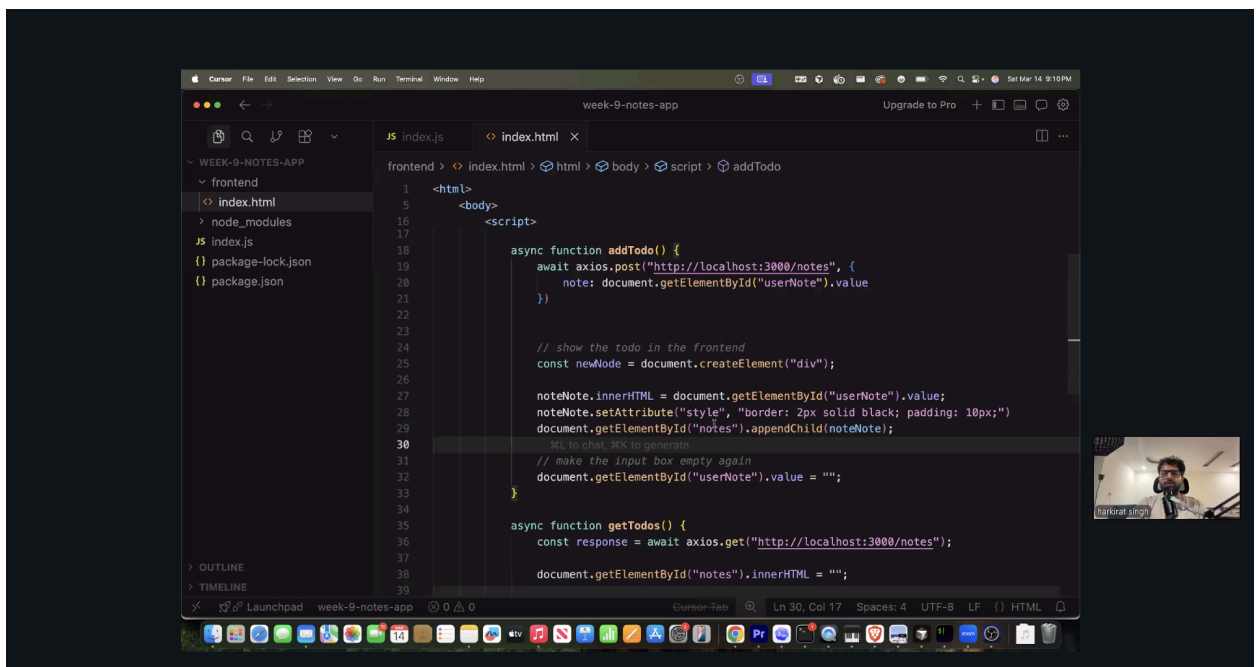


The screenshot shows a code editor with a file explorer on the left. The file explorer shows a project named 'WEEK-9-NOTES-APP' with a 'frontend' directory containing 'index.html'. The main editor area shows the 'index.html' file with the following code:

```
1 <html>
5 <body>
16 <script>
33 }
34
37 async function getTodos() {
38   const response = await axios.get("http://localhost:3000/notes");
39   const notes = response.data.notes;
40
41   document.getElementById("notes").innerHTML = "";
42
43   for (let i = 0; i < notes.length; i++) {
44     const noteNote = document.createElement("div");
45     noteNote.innerHTML = notes[i];
46     noteNote.setAttribute("style", "border: 2px solid black; padding: 10px;");
47     document.getElementById("notes").appendChild(noteNote);
48   }
49
50   getTodos();
51 }
52 </script>
53 </body>
54 </html>
```

The code defines an asynchronous function 'getTodos' that uses 'axios.get' to fetch data from 'http://localhost:3000/notes'. It then iterates over the 'notes' array and creates 'div' elements for each note, setting their innerHTML and style, and appends them to the 'notes' container. The function is called at the end of the script.

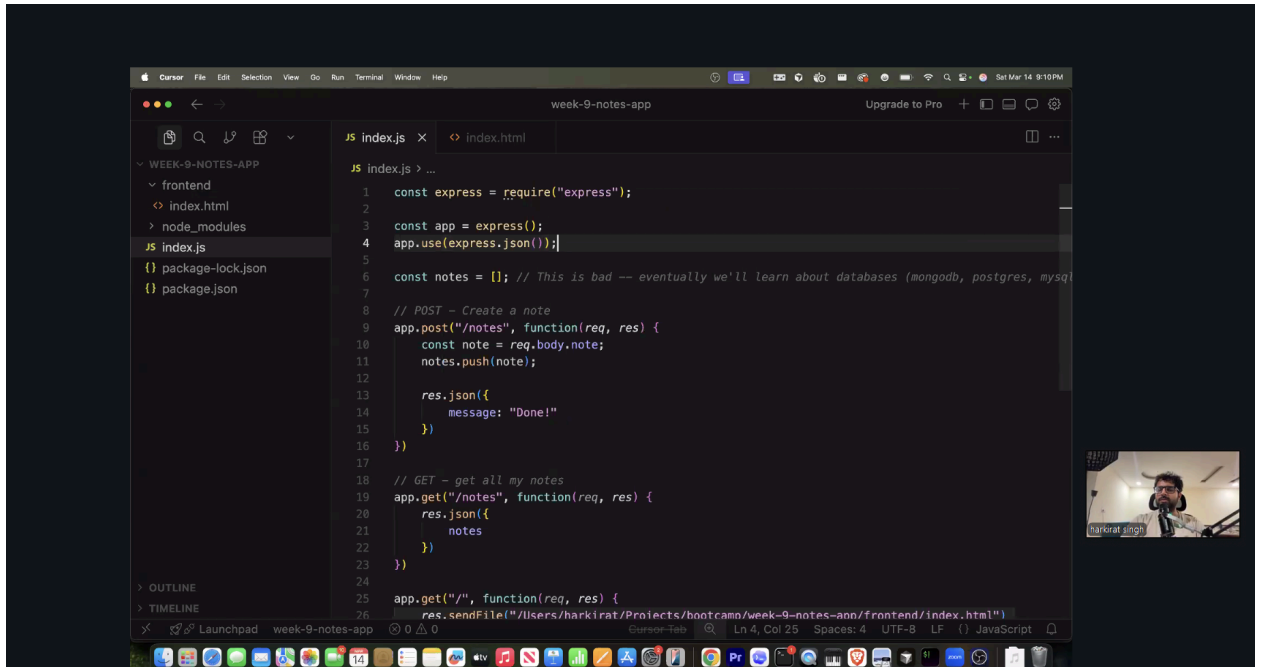
- Add todo frontend code(POST request)



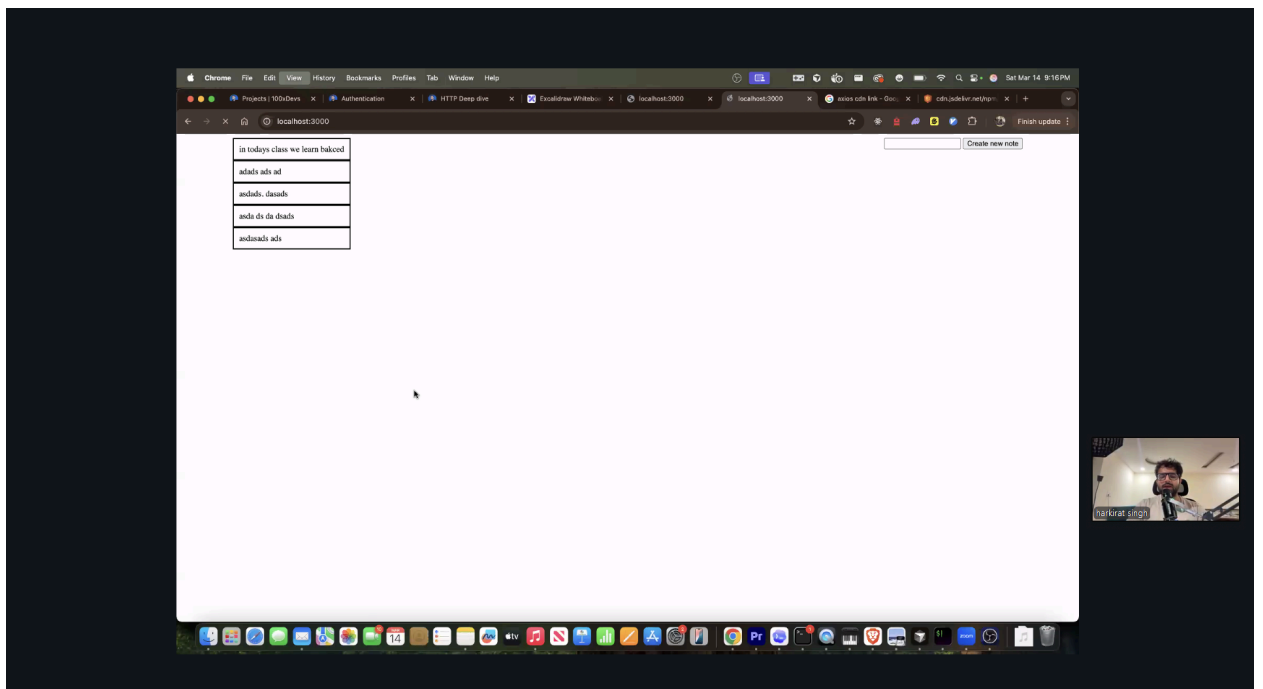
The screenshot shows a code editor with a file explorer on the left. The file explorer shows a project named 'WEEK-9-NOTES-APP' with a 'frontend' directory containing 'index.html'. The main editor area shows the 'index.html' file with the following code:

```
1 <html>
5 <body>
16 <script>
17
18 async function addTodo() {
19   await axios.post("http://localhost:3000/notes", {
20     note: document.getElementById("userNote").value
21   });
22
23   // show the todo in the frontend
24   const newNode = document.createElement("div");
25
26   noteNote.innerHTML = document.getElementById("userNote").value;
27   noteNote.setAttribute("style", "border: 2px solid black; padding: 10px;");
28   document.getElementById("notes").appendChild(noteNote);
29
30   // make the input box empty again
31   document.getElementById("userNote").value = "";
32 }
33
34
35 async function getTodos() {
36   const response = await axios.get("http://localhost:3000/notes");
37
38   document.getElementById("notes").innerHTML = "";
39 }
```

The code defines an asynchronous function 'addTodo' that uses 'axios.post' to send data to 'http://localhost:3000/notes'. The data is an object with a 'note' property whose value is taken from the 'userNote' input field. After the post request, it creates a new 'div' element, sets its innerHTML to the value of 'userNote', sets its style, and appends it to the 'notes' container. It also clears the 'userNote' input field. The 'getTodos' function is also present, as seen in the previous screenshot.



When we reload the page

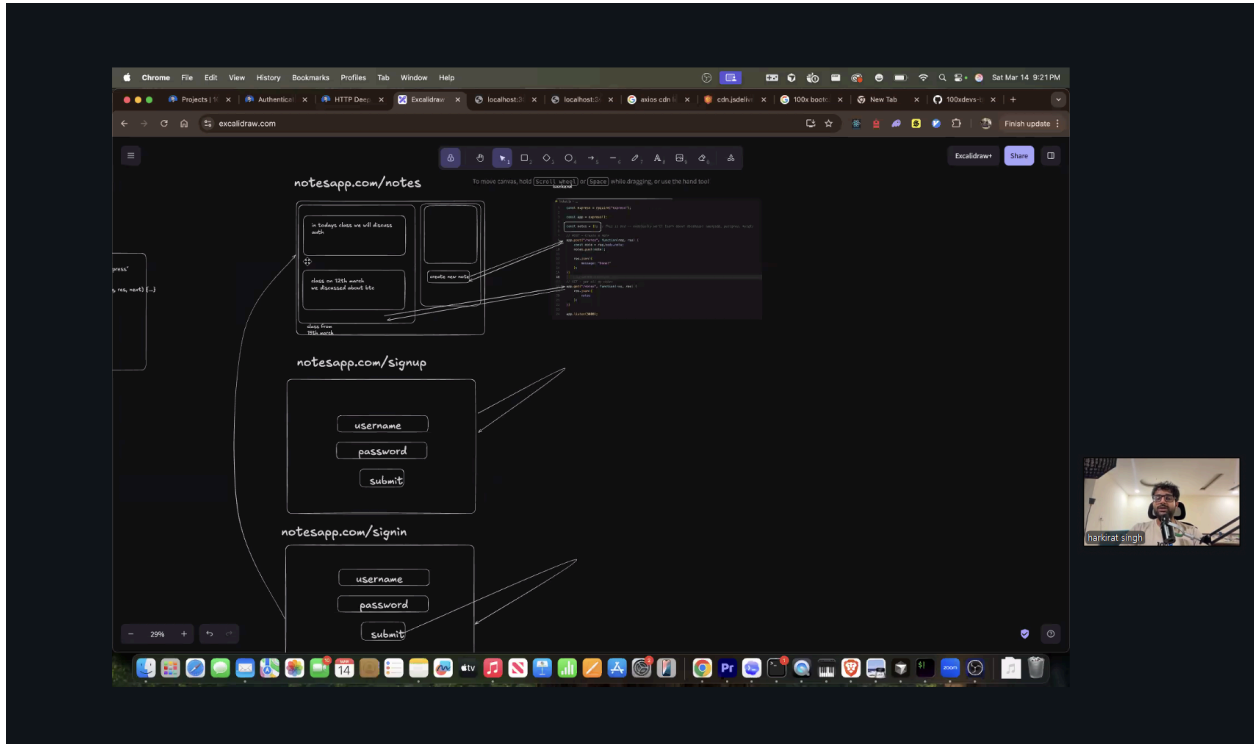


9. Lets add Authentication

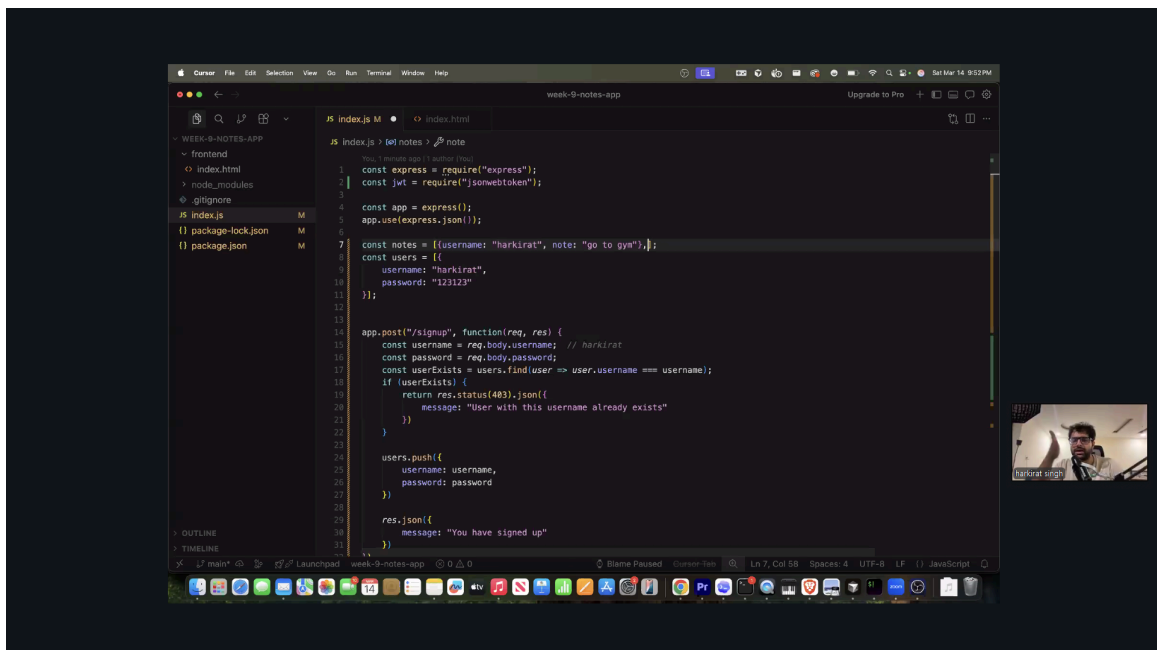
Step1: Sign-up page

Step2: Sign-in page

Step3: todo page



10. Lets add signup route



11. Lets make sign-in route

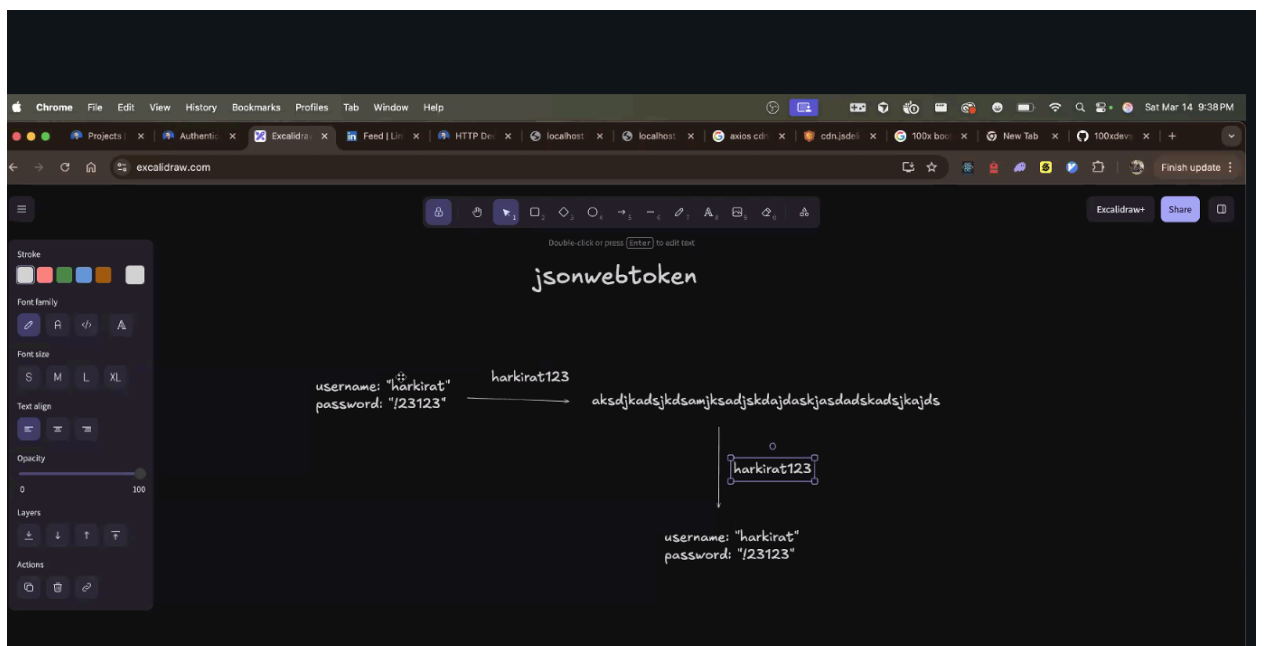
It is tricky so we during sign-in token and that token is store somewhere in browser

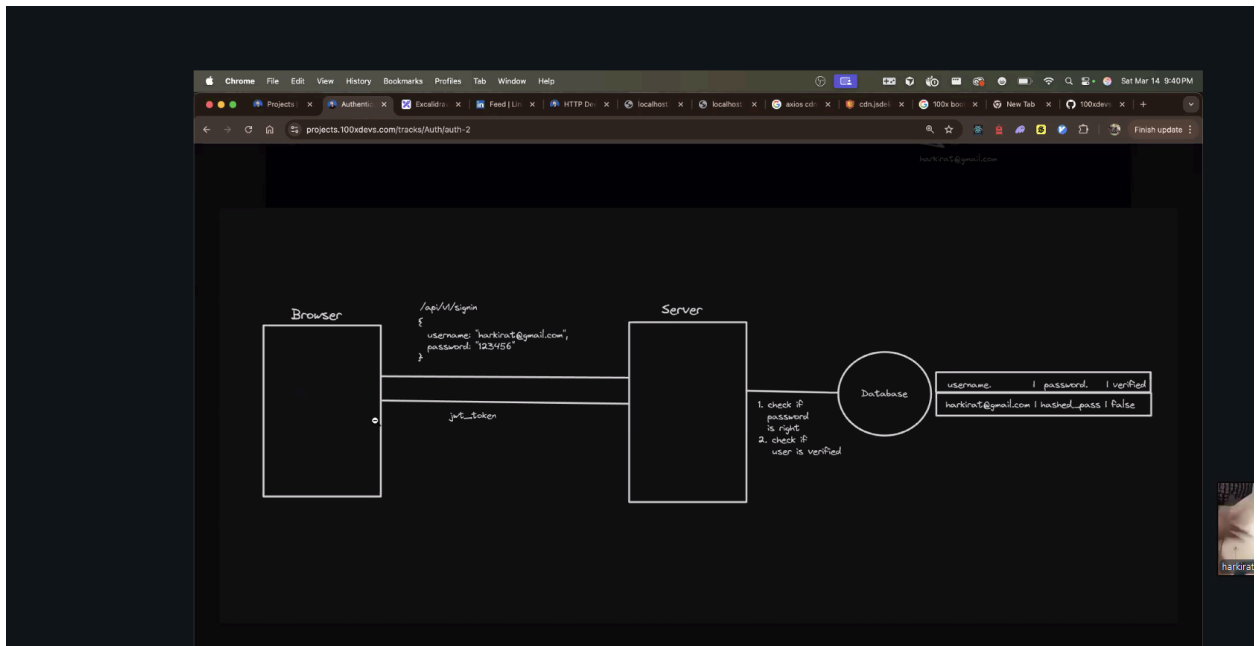
Token is created using json web token

```
JS index.js M X <> index.html

JS index.js > [🔗] jwt

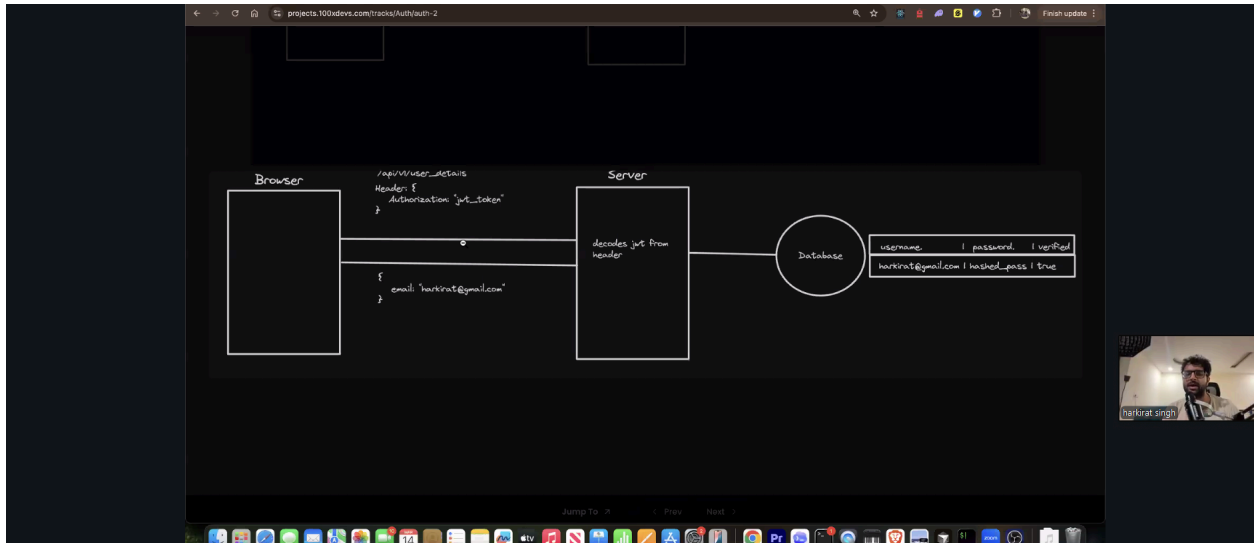
    You, now | 1 author (You)
1   const express = require("express");
2   const jwt = require("jsonwebtoken");
3
4   const app = express();
```





Secret only know by developers and this secret never exposed in frontend.

```
32 }
33
34 app.post("/signin", function(req, res) {
35   const username = req.body.username;
36   const password = req.body.password;
37
38   const userExists = users.find(user => user.username === username && user.password === password);
39
40   if (!userExists) {
41     res.status(403).json({
42       message: "Incorrect credentials"
43     });
44     return;
45   }
46
47   // json web tokens
48   const token = jwt.sign({
49     username: username
50   }, "harkirat123");
51
52   res.json({
53     token: token
54   });
55 })
56
57 // POST - Create a note
58 app.post("/notes", function(req, res) {
```

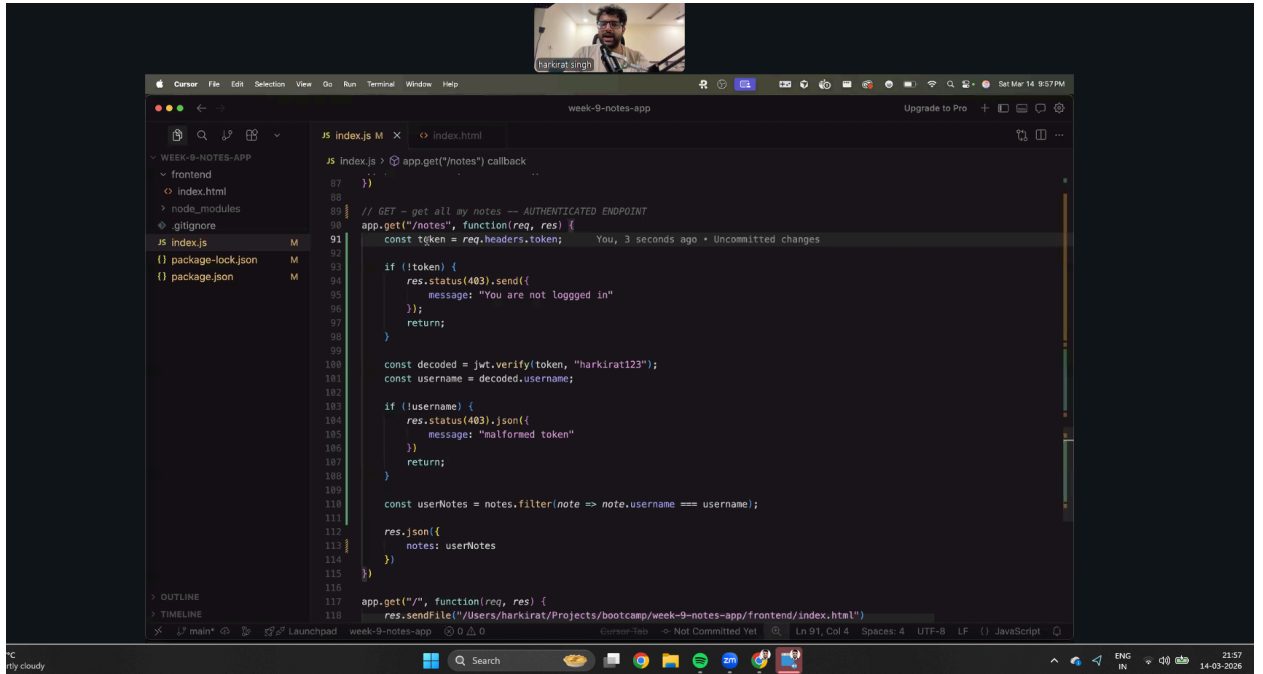



Before user create notes or render notes in frontend they have to go through authentication

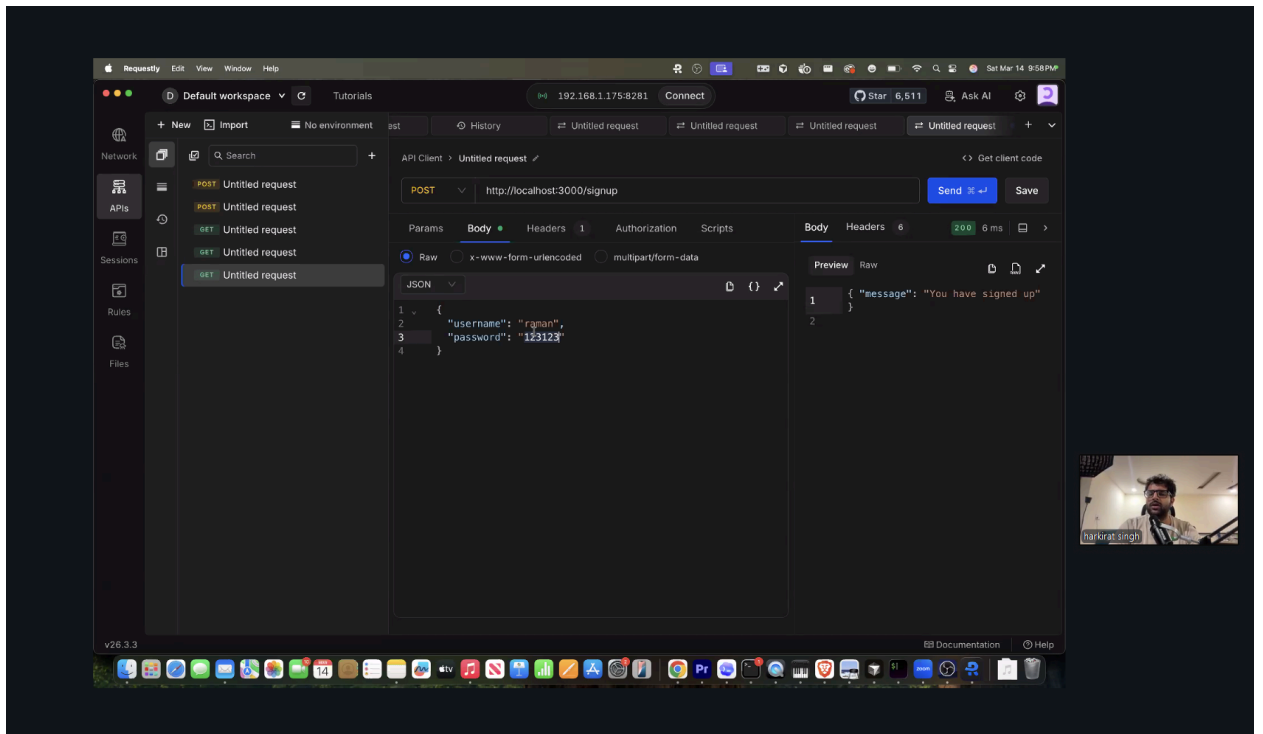
The screenshot shows a code editor with the following code in `index.js`:

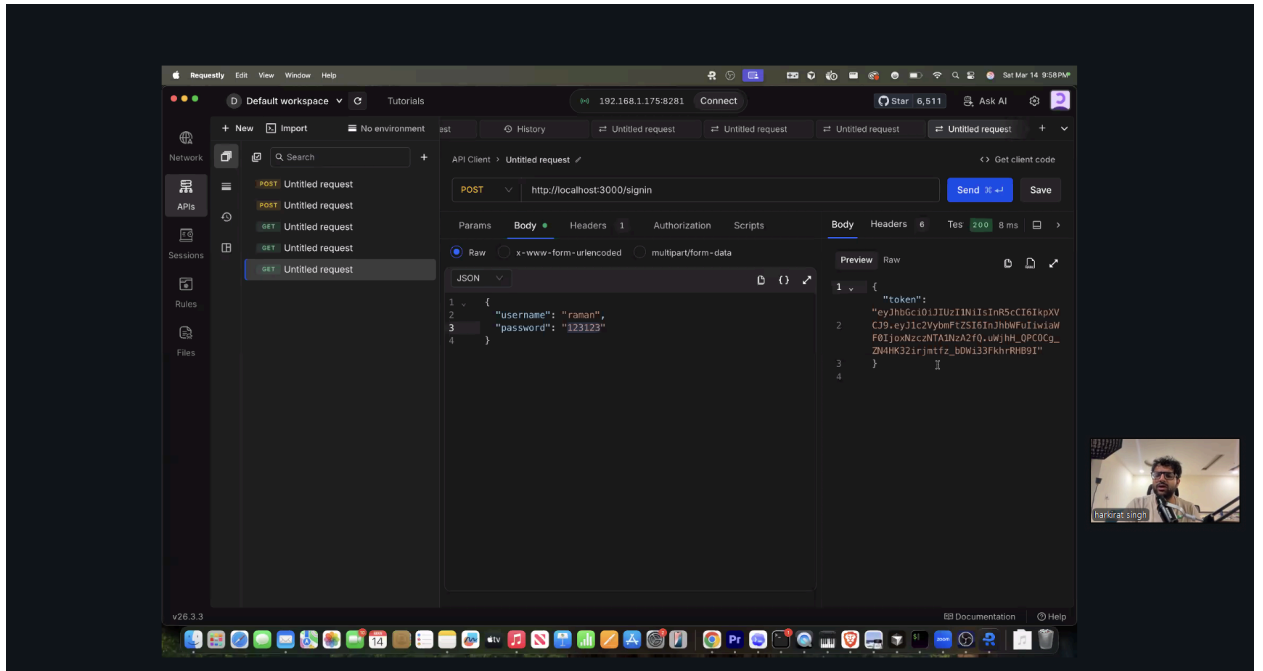
```
57 // POST - Create a note - AUTHENTICATED ENDPOINT
58 app.post("/notes", function(req, res) {
59   // check if they have sent the right header, extract who this user is from the header.
60   const token = req.headers.token;
61
62   if (!token) {
63     res.status(403).send({
64       message: "You are not logged in"
65     });
66     return;
67   }
68
69   const decoded = jwt.verify(token, "harkirat123");
70   const username = decoded.username;
71
72   if (!username) {
73     res.status(403).send({
74       message: "malformed token"
75     });
76     return;
77   }
78
79   const note = req.body.note;
80   notes.push(note);
81
82   res.json({
83     message: "Done!"
84   });
85 }
86
87
88
```

The code implements a POST endpoint for creating a note. It checks if the user is authenticated by verifying the JWT token. If the token is valid, it pushes the note to the database and returns a success message. If the token is invalid or missing, it returns a 403 status with an error message.

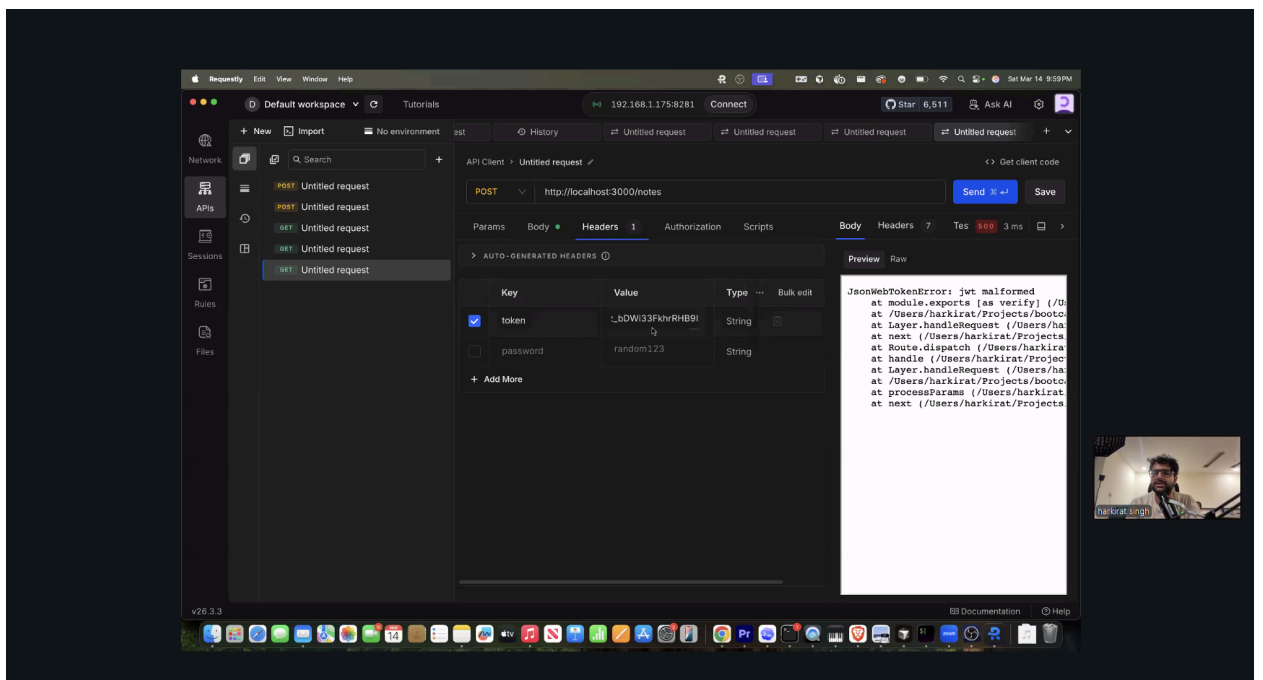


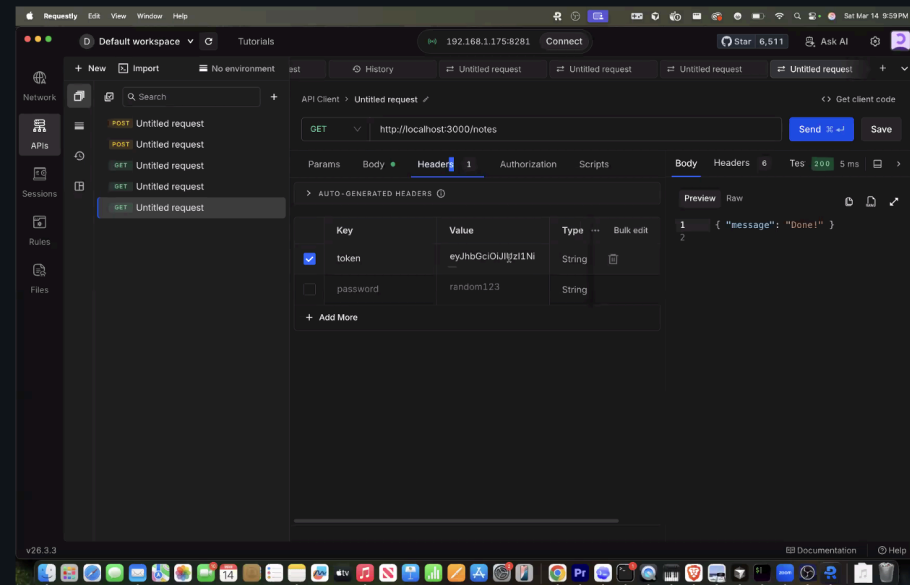
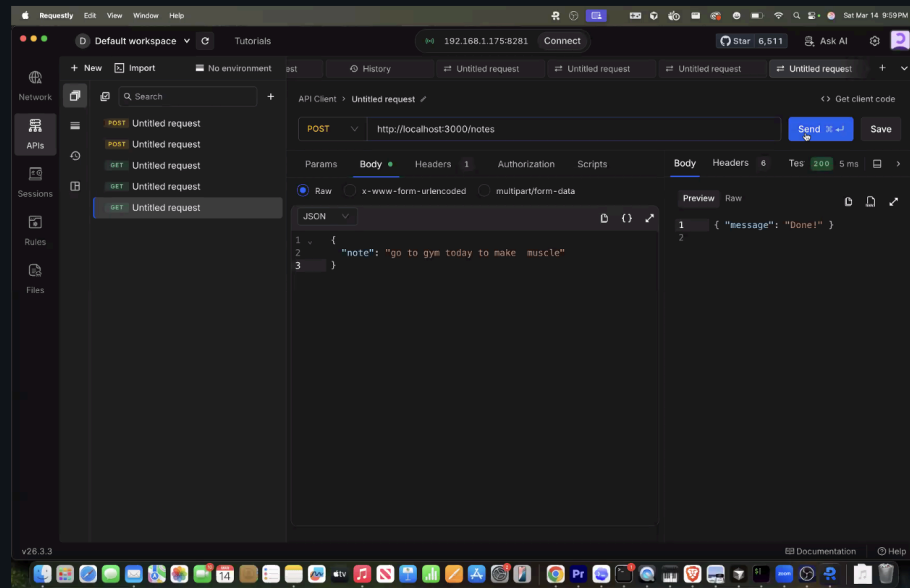
Now Lets test

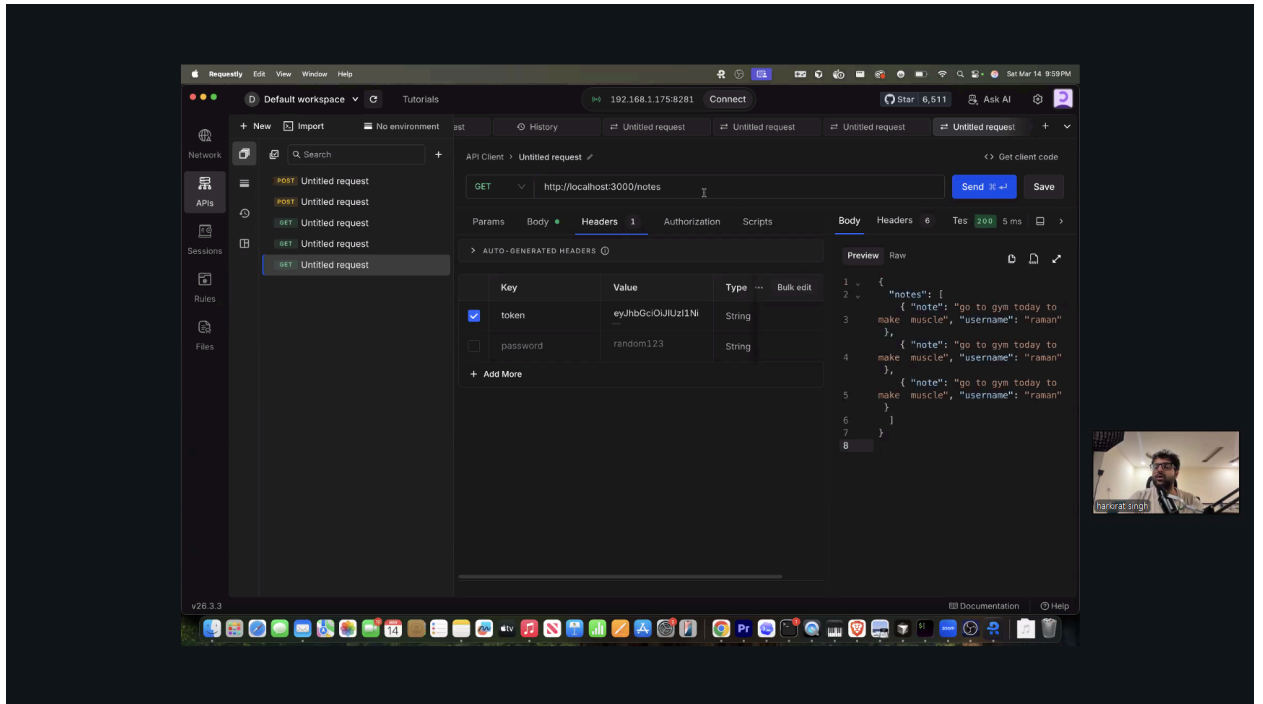




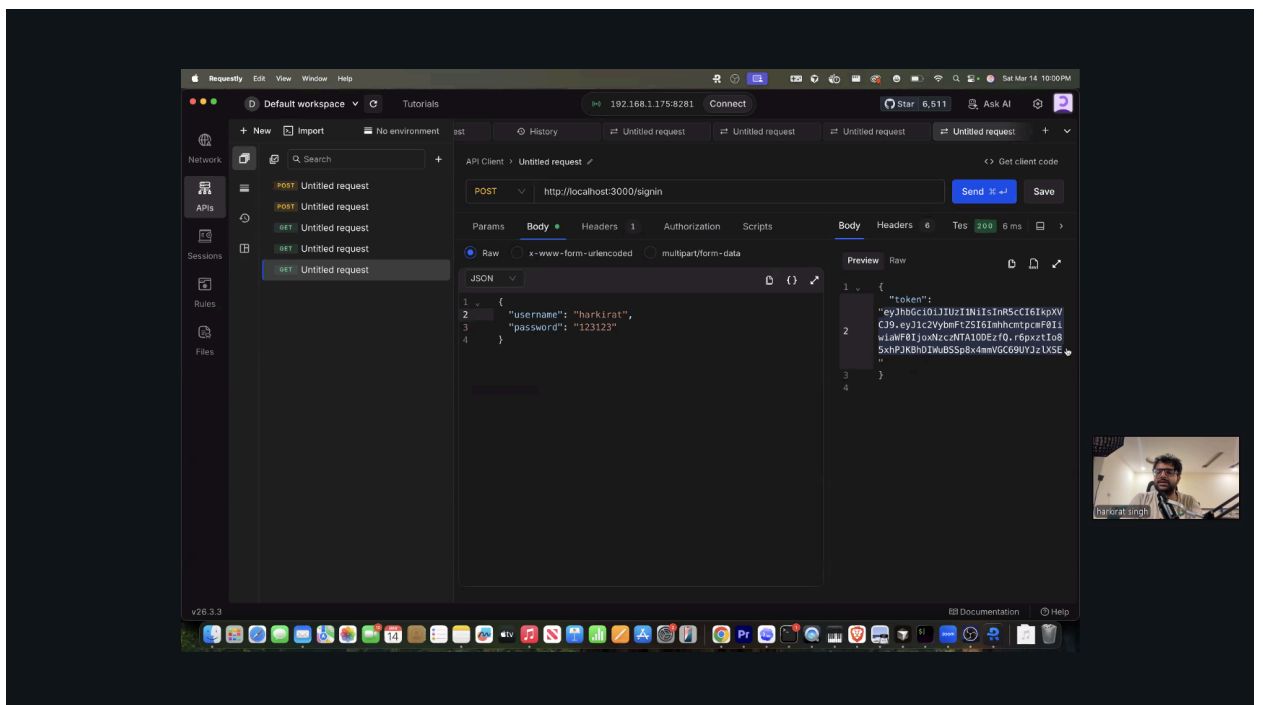
If I given any random token

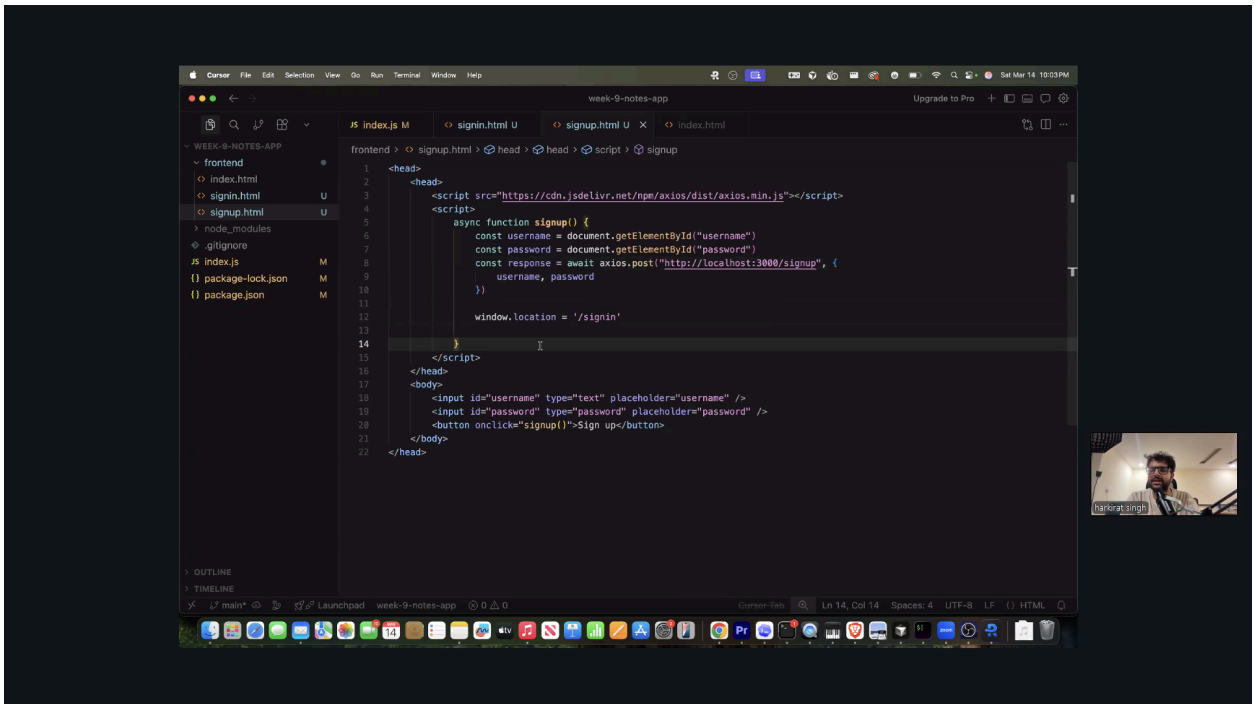
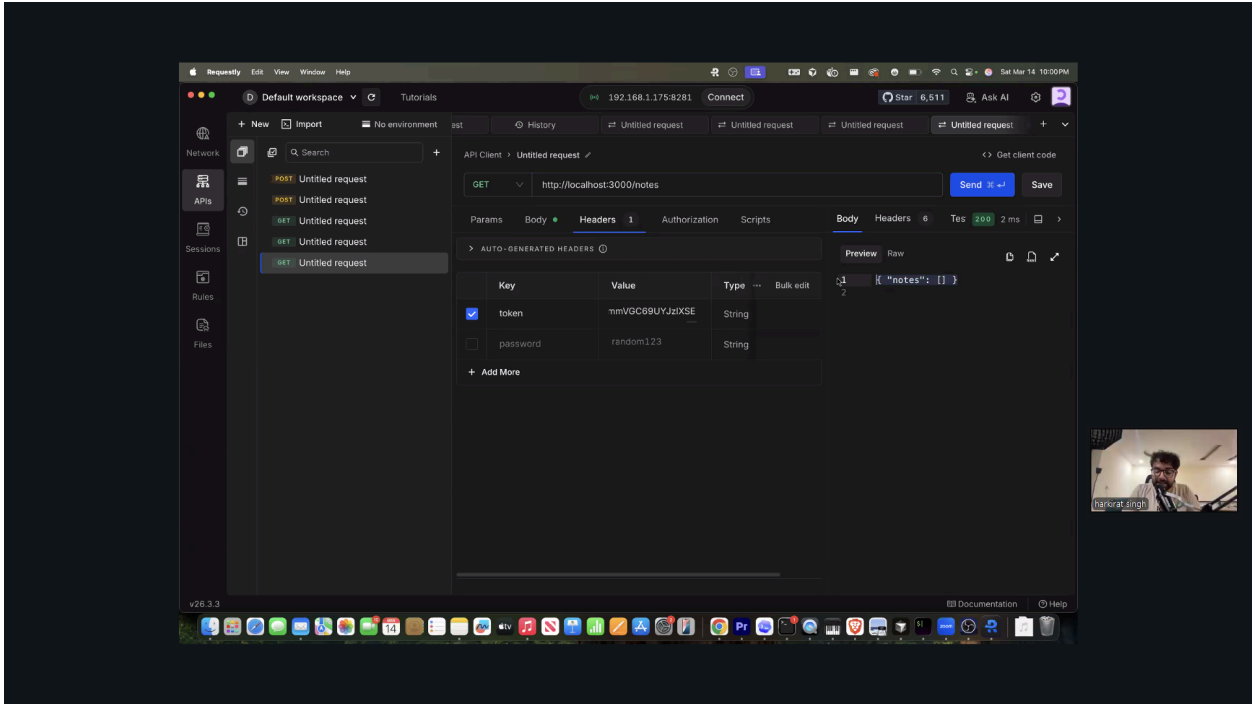


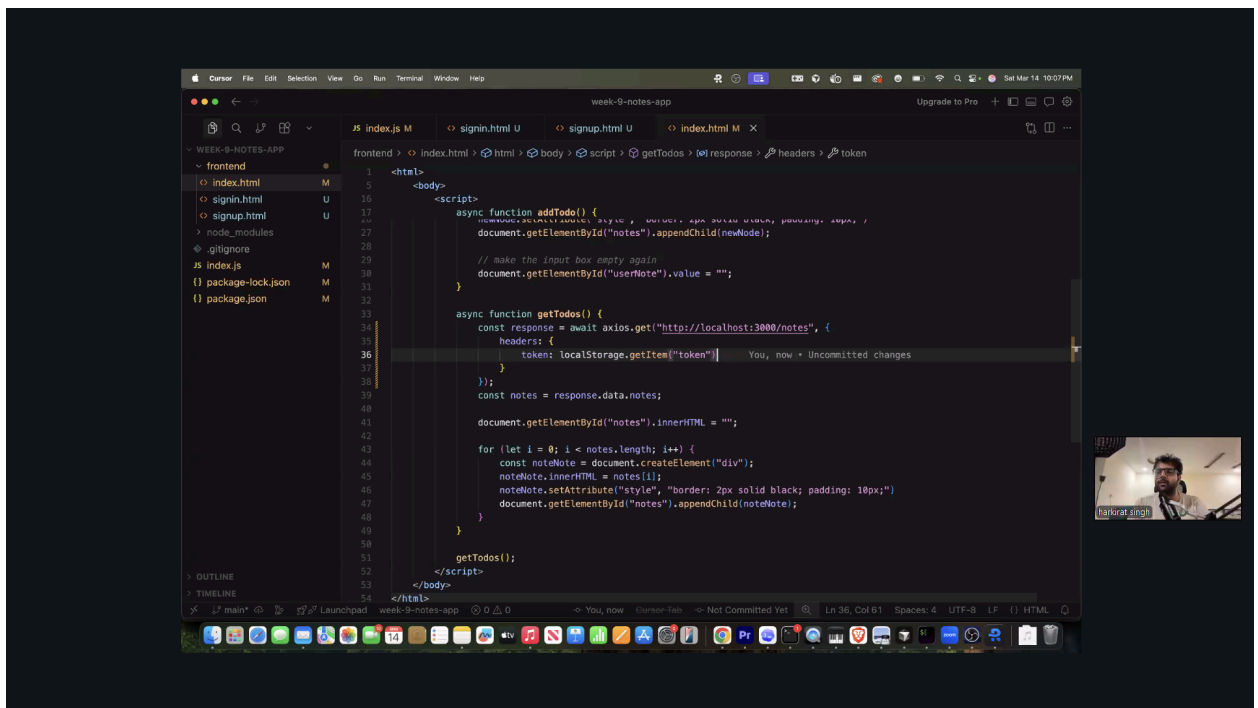
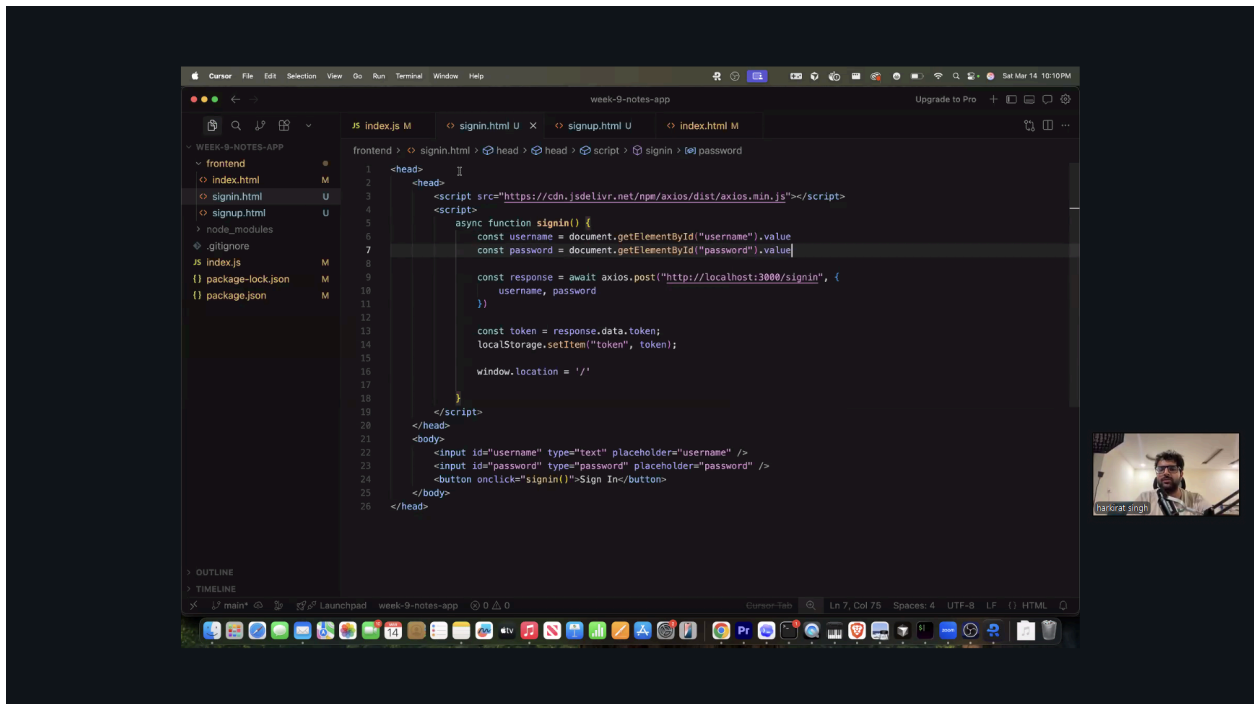




Let's sign-in with another user







Chrome DevTools Performance tab showing a timeline of requests for a page loaded from localhost:3000. The page is a simple white page with a "Create new note" button.

The Performance tab shows a timeline of requests. The "Overview" section is selected, showing a timeline of requests from 0 to 260 ms. The "Network" tab is also visible, showing a list of requests.

Name	Status	Type	Initiator	Size	Time
signin	200	xhr		0.4 kB	5 ms
localhost	200	document		2.4 kB	4 ms
axios.min.js	200	script	:3000/:3	(memory cache)	0 ms
inpage-evm-early.js	200	script	contentScript-early-evm.js:1	3.6 kB	2 ms
injected.js	200	script	contentScript.js:1	7,332 kB	187 ms
inpage.js	200	script	content.js:2	307 kB	1 ms
notes	200	xhr	xhr.js:220	0.2 kB	4 ms

7 requests | 7.6 MB transferred | 7.7 MB resources | Finish: 235 ms | DOMContentLoaded: 228 ms | Load: 659 ms

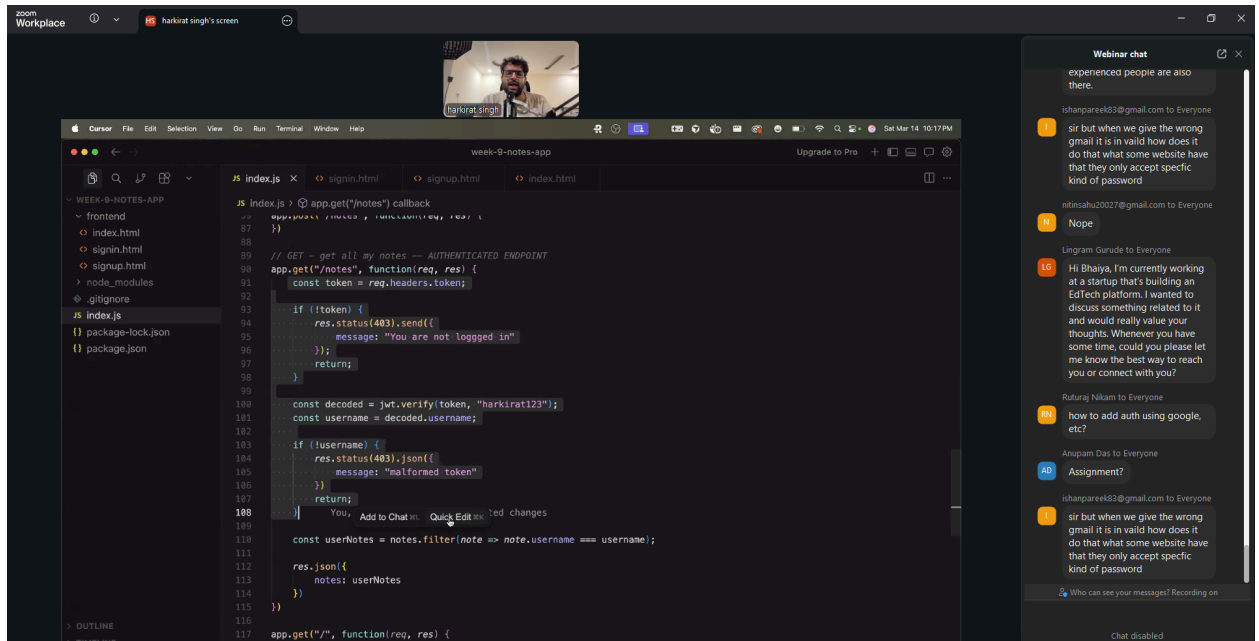
Chrome DevTools Application tab showing the state of the application for http://localhost:3000. The page is a simple white page with a "Create new note" button.

The Application tab shows the state of the application. The "Storage" section is selected, showing a list of storage items. The "Local storage" section is expanded, showing a list of items.

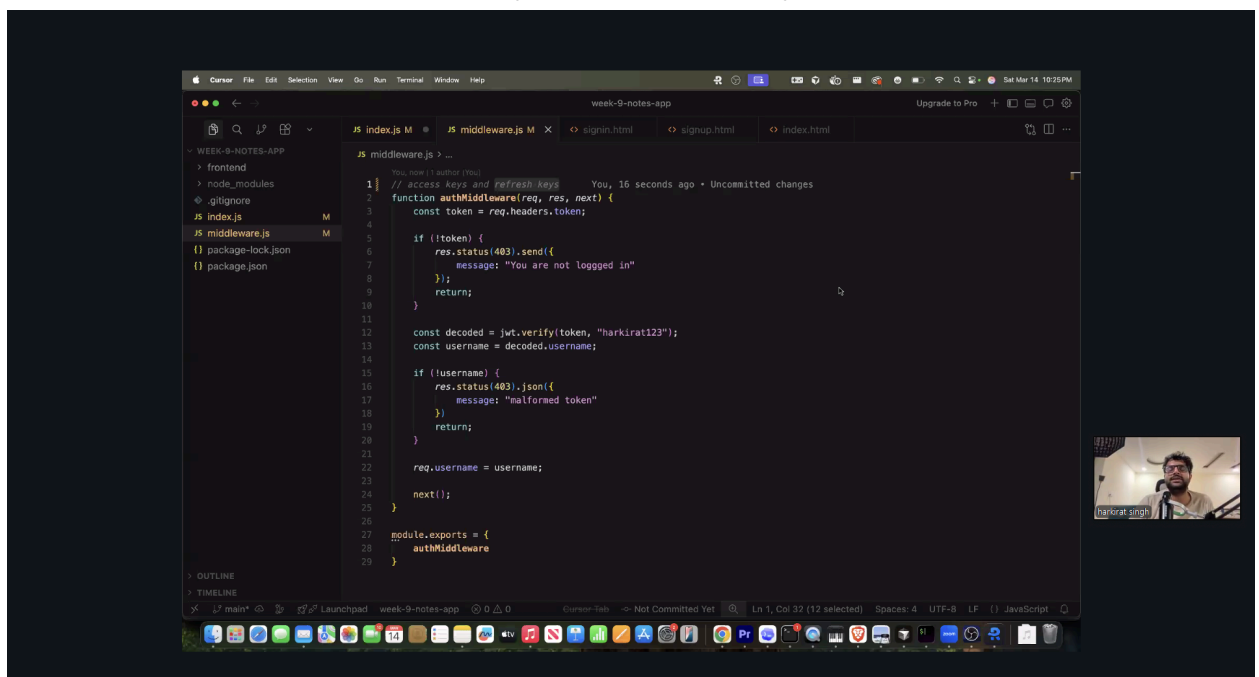
Key	Value
ally-supports-cache	("userAgent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) Appl...
nextauth.message	("event":"session","data":{"trigger":"getSession"},"timestamp":177...
remotion.editor.marksv2	{}
remotion.editor.timelineFlex.top-to-bottom	0.7299471830985915
remotion.lastCrashBecauseOfHooks	1773352770058
remotion.playbackrate	1
remotion.sidebarPanel	compositions
remotion.time-all	("RustScheduler":2000,"LifetimeAnnotations":0)
remotion.zoom-map	{}
token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImhhbG...
user	{"id":"4","email":"harkirat22@gmail.com","name":"harkirat","role":"us...

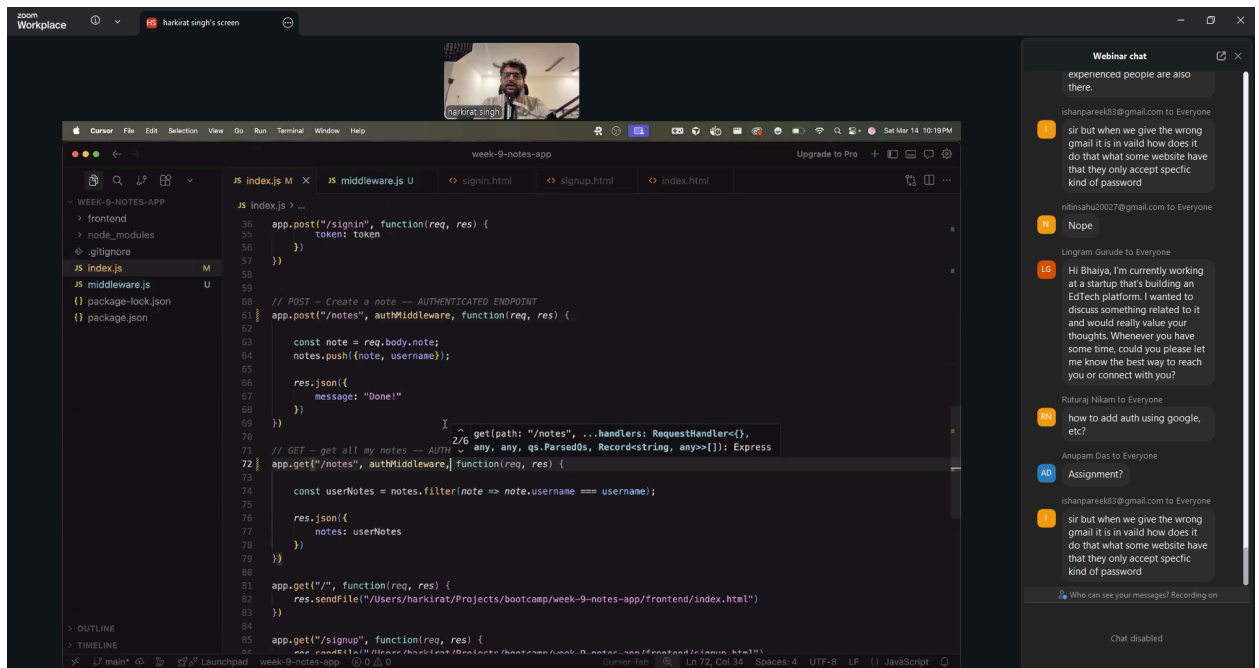
1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImhhbG...

12. Auth Middleware(Remove repetitive code)

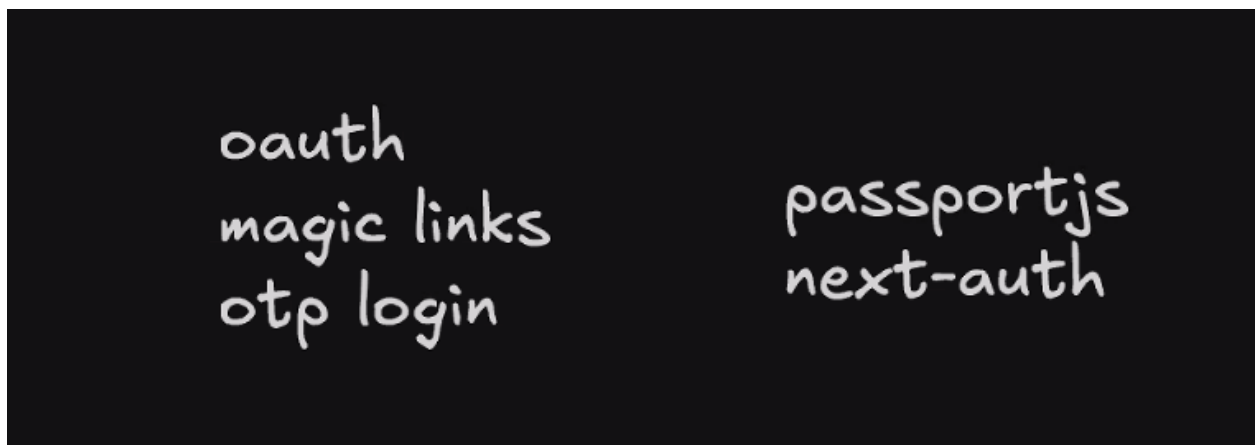


Read about concept access key and refresh key





Other type of sign-in



Assignment:

